Marketly

Microservices-Based E-Commerce Platform

Project Documentation

Marketly is a microservices-based e-commerce platform designed to provide a scalable and reliable online shopping experience.

Item	Details
Team members	سيف الدين محمود محمد ناجي عبدالعليم عمر محمد يوسف فرج سوتير أسامة ثروت لطيف معاذ محمد السيد محمد محمد صلاح محمد محمد محمد ممدوح المعصراوي شعبان
Repository	https://github.com/Seif-El-Din-Mahmoud/marketly

Table of Contents

1. Project Planning & Management 3
   1.1 Project Proposal 3
      Overview 3
      Problem Statement 3
      Objectives 3
      Scope 3
   1.2 Project Plan 4
      Timeline (Gantt Chart) 4
      Milestones 4
      Deliverables 5
      Resource Allocation 5
   1.3 Task Assignment & Roles 5
   1.4 Risk Assessment & Mitigation Plan 6
   1.5 KPIs (Key Performance Indicators) 7
2. Literature Review 8
   2.1 Similar Projects and Solutions 8
   2.2 Comparison with Marketly 8
   2.3 Lessons Learned and Ideas Adopted 9
   2.4 Feedback & Evaluation, Improvements, and Grading 9
      Feedback & Evaluation 9
      Suggested Improvements 9
      Final Grading Criteria 10
   2.7 References 10
3. Requirements Gathering 11
   3.1 Stakeholder Analysis 11
   3.2 User Stories & Use Cases 11
      User Stories 11
      Use Cases 12
   3.3 Functional Requirements 12
   3.4 Non-functional Requirements 13

Note: if the table above appears empty, right-click it in Word and choose "Update Field".

1. Project Planning & Management

1.1 Project Proposal

Overview

Marketly is a microservices-based e-commerce platform designed to provide a scalable and reliable online shopping experience. It allows users to browse and search for products, manage their shopping cart, place orders, and complete payments through a set of independent services.

The platform is designed around containerization, automated deployment, scalability, monitoring, and infrastructure automation. Each business capability (users, catalog, cart, orders, payments) runs as a separate, independently deployable service, so that parts of the system can be scaled, updated, or fixed without affecting the rest.

Problem Statement

Traditional monolithic e-commerce applications are difficult to scale under traffic spikes, risky to deploy (a single change requires redeploying everything), and a single failure can bring down the entire store. Small and medium businesses need a platform that stays available during peak demand while remaining easy to deploy and maintain.

Objectives

- Design and implement an e-commerce platform as a set of independent microservices.
- Containerize every service with Docker for consistent environments from development to production.
- Automate build, test, and deployment through a CI/CD pipeline.
- Support horizontal scaling and self-healing through container orchestration (Kubernetes).
- Provision infrastructure through code (Infrastructure as Code) so environments are reproducible.
- Provide centralized monitoring, logging, and alerting for system health and performance.
- Document the architecture, requirements, testing, and deployment so the project is reproducible.

Scope

In scope:

- User registration, authentication, and profile management.
- Product catalog: browsing, product details, categories, and search.
- Shopping cart: add, update, remove items.
- Order placement and order history/status tracking.
- Payment processing through a payment gateway (sandbox/test mode).
- Containerization, CI/CD, orchestration, IaC, and monitoring of all services.

Out of scope:

- Real-money payment processing and PCI-DSS certification (sandbox gateway only).
- Native mobile applications (responsive web UI only).

- Advanced features such as AI recommendations, multi-vendor marketplaces, and logistics/shipping integration.

1.2 Project Plan

Timeline (Gantt Chart)

The project is planned over 8 weeks. Dark cells show the weeks in which each activity is active.

Task	W1	W2	W3	W4	W5	W6	W7	W8
Requirements & planning	■							
Architecture & design	■	■						
Core services (User, Product, Search)		■	■	■				
Cart, Order, Payment services			■	■	■			
Frontend (web UI)			■	■	■	■		
Containerization (Docker)		■	■	■				
CI/CD pipeline				■	■	■		
Kubernetes deployment & scaling					■	■	■	
Infrastructure as Code					■	■	■	
Monitoring & logging						■	■	
Testing (unit/integration/load)					■	■	■	
Documentation	■	■	■	■	■	■	■	■
Final demo & presentation								■

Milestones

ID	Milestone	Week	Completion Criteria
M1	Project kickoff & requirements approved	1	Proposal, stakeholder analysis, and requirements agreed by the team.
M2	Architecture & design complete	2	Service boundaries, API contracts, and data models defined.
M3	Core services working	4	User, Product, and Search services run locally with passing unit tests.
M4	Transaction flow complete	5	Cart → Order → Payment works end to end.
M5	Automated delivery in place	6	CI/CD builds, tests, and deploys containers; frontend connected.
M6	Orchestration, IaC & monitoring ready	7	Kubernetes deployment with autoscaling, dashboards and alerts live; infrastructure created from code.
M7	Testing complete	7	Functional, integration, and load tests executed and reported.
M8	Final delivery	8	Demo, final documentation, and presentation delivered.

Deliverables

Deliverable	Description	Due
Project documentation	This document (planning, literature review, requirements).	Week 1 (draft), Week 8 (final)
Source code	All microservices, frontend, Dockerfiles, and configuration in GitHub.	Week 7
CI/CD pipeline	Automated build, test, and deployment workflows.	Week 6
Infrastructure code	IaC and Kubernetes manifests/Helm charts.	Week 7
Monitoring dashboards	Metrics, logs, and alert rules.	Week 7
Test report	Test plan, results, and load-test findings.	Week 7
Final presentation & demo	Live demonstration and slides.	Week 8

Resource Allocation

Resource Type	Resource	Purpose
Human	6 team members (see Section 1.3)	Design, development, DevOps, testing, documentation.
Version control	GitHub repository	Source code, documentation, issue tracking, project board.
CI/CD	GitHub Actions	Automated build, test, and deploy.
Containers	Docker, Docker Compose	Packaging and local development.
Orchestration	Kubernetes (local cluster or cloud-managed)	Deployment, scaling, self-healing.
Infrastructure	Terraform / cloud free-tier or university lab	Reproducible infrastructure.
Monitoring	Prometheus, Grafana, centralized logging	Metrics, dashboards, alerting.
Databases	PostgreSQL / MongoDB (per service)	Persistent data for each service.
Time	8 weeks	Full project duration.

1.3 Task Assignment & Roles

Role	Team Member	Responsibilities
Project Manager / FrontEnd/DevOps	Seif El Din Mahmoud	Planning, schedule tracking, team meetings, risk management, presentation along with Web user Interface development and DevOps tasks.
Frontend Developer	Omar Mohamed	Web user interface: product browsing, search, cart, checkout, and order pages.
Backend Developer/ DevOps	Sotir Usama	User/authentication, Product, and Search services; REST APIs; database design.
Backend Developer/DevOps	Moaz Mohamed	Cart, Order, and Payment services; inter-service communication; payment gateway integration.

Role	Team Member	Responsibilities
DevOps Engineer 1	Mohamed Mamdouh	Docker, CI/CD pipeline, Kubernetes deployment and autoscaling.
DevOps / QA Engineer	Mohamed Salah	Infrastructure as Code, monitoring and logging, test planning and execution (functional, integration, load).

Responsibility matrix (R = Responsible, S = Supports):

Activity	M1	M2	M3	M4	M5	M6
Requirements & planning	R	S	S	S	S	S
Architecture & API design	S	S	R	R	S	S
Frontend development	S	R	S	S		
Backend services	S	S	R	R		
Containerization & CI/CD			S	S	R	S
Kubernetes, IaC, monitoring					R	R
Testing	S	S	S	S	S	R
Documentation & presentation	R	S	S	S	S	S

1.4 Risk Assessment & Mitigation Plan

Likelihood (L) and Impact (I) are rated Low, Medium, or High.

#	Risk	L	I	Mitigation
1	Service integration problems (API mismatches between services)	High	High	Define API contracts early (OpenAPI), use contract tests, and integrate continuously.
2	Team member unavailable or workload imbalance	Medium	High	Assign a backup for each role, keep documentation current, review the task board weekly.
3	Underestimated complexity of Kubernetes/IaC within the 8-week timeline	Medium	High	Start with Docker Compose, move to Kubernetes incrementally, allocate learning time.
4	Single service failure causing cascading failures	Medium	High	Timeouts, retries, circuit breakers, health checks, and Kubernetes self-healing.
5	Security vulnerabilities (leaked secrets, weak authentication)	Medium	High	Use secret management, JWT with expiry, HTTPS, dependency scanning in CI.
6	Payment gateway sandbox unavailable or changes API	Low	Medium	Abstract the gateway behind an interface and provide a mock payment provider.
7	Performance degradation under load	Medium	Medium	Load testing, caching, autoscaling, and database indexing.
8	Cloud/infrastructure cost overrun or lab access limits	Medium	Medium	Use free tiers or local clusters; tear down environments after use via IaC.
9	Scope creep	Medium	Medium	Keep the scope in Section 1.1 as the baseline; log new ideas as future work.

#	Risk	L	I	Mitigation
10	Data loss	Low	High	Regular database backups and persistent volumes.

1.5 KPIs (Key Performance Indicators)

KPI	Description	Target	Measured By
API response time	95th-percentile response time for main endpoints.	< 500 ms	Prometheus / load test
System uptime	Percentage of time the platform is available.	≥ 99.5%	Monitoring / health checks
Error rate	Share of requests that return 5xx errors.	< 1%	Prometheus / logs
Deployment frequency	How often new versions are deployed automatically.	Every merge to main	CI/CD pipeline
Deployment time	Time from merge to running in the cluster.	< 10 minutes	CI/CD pipeline
Scalability	Concurrent users handled with autoscaling enabled.	≥ 500 concurrent users	Load test (k6/JMeter)
Test coverage	Code covered by automated tests.	≥ 70%	Coverage report in CI
Order success rate	Orders completed successfully after payment.	≥ 98%	Order/Payment service metrics
User adoption rate	Registered users who place at least one order during the demo/pilot.	≥ 60% of pilot users	Database analytics / pilot survey
Recovery time	Time to recover a crashed service.	< 1 minute	Kubernetes events

2. Literature Review

This chapter reviews projects and solutions similar to Marketly, identifies their advantages and disadvantages, and compares them with Marketly to show the differences and the ideas that can be reused in its development. The sources used are documented in Section 2.7.

2.1 Similar Projects and Solutions

Solution	Description	Advantages	Disadvantages
Shopify	Hosted SaaS e-commerce platform for merchants of all sizes.	Very easy to set up; reliable hosting; large app ecosystem; built-in payments and themes.	Subscription and transaction fees; limited control over the backend and architecture; vendor lock-in.
Magento (Adobe Commerce)	Open-source, highly customizable e-commerce platform, traditionally a monolith.	Very flexible and feature-rich; large community; supports complex catalogs.	Heavy and resource-intensive; complex to deploy and maintain; scaling requires considerable effort.
WooCommerce	E-commerce plugin for WordPress.	Low cost; quick to launch; huge plugin ecosystem.	Monolithic and tied to WordPress; performance and security depend on plugins; limited horizontal scalability.
Medusa	Open-source headless commerce engine written in Node.js with a modular architecture.	Modular and API-first; developer-friendly; easy to customize.	Younger ecosystem; deployment and infrastructure are left to the user; typically deployed as a single backend rather than many independent services.
Microsoft eShopOnContainers	Reference .NET microservices e-commerce application using Docker and Kubernetes.	Clear microservices patterns (API gateway, event bus, CQRS); good documentation; container-based deployment.	Reference/learning project, not production-ready; tied to the .NET stack; no focus on full IaC or CI/CD automation; the project is no longer actively developed.
Sock Shop (Weaveworks)	Demo microservices application for a sock store, built with multiple languages.	Polyglot services; widely used for demonstrating monitoring, tracing, and load testing.	Minimal business features (no real order/payment logic); no longer actively maintained.

2.2 Comparison with Marketly

Criterion	Shopify	Magento / Woo	Medusa	eShopOn-Containers / Sock Shop	Marketly
Independent microservices	No (hidden)	No (monolith)	Partial (modules)	Yes	Yes
Containerized	N/A (SaaS)	Optional	Optional	Yes	Yes
Automated CI/CD	Managed by vendor	Manual setup	Manual setup	Limited	Yes (built-in)

Criterion	Shopify	Magento / Woo	Medusa	eShopOn-Containers / Sock Shop	Marketly
Kubernetes autoscaling	Managed by vendor	Difficult	Possible	Basic	Yes
Infrastructure as Code	No	No	No	Limited	Yes
Monitoring & alerting	Vendor dashboards	Add-ons	Add-ons	Partial	Yes (Prometheus/Grafana)
Full control of architecture	No	Yes	Yes	Yes	Yes
Licensing cost	Paid	Free/Paid	Free	Free	Free

Key differences: unlike hosted or monolithic platforms, Marketly splits the store into independent services that can be deployed and scaled separately. Compared with reference projects such as eShopOnContainers and Sock Shop, Marketly adds a complete DevOps layer: automated CI/CD, infrastructure as code, and integrated monitoring, which these demos largely leave out.

2.3 Lessons Learned and Ideas Adopted

- **API Gateway pattern** (from eShopOnContainers): a single entry point for clients that routes requests to the correct service.
- **Database per service** (microservices best practice): each service owns its data to keep services loosely coupled.
- **Observability from day one** (from Sock Shop): metrics, logs, and tracing are included from the start, not added later.
- **Headless, API-first design** (from Medusa): the frontend consumes REST APIs, so clients can change independently.
- **Avoiding plugin dependency** (from WooCommerce/Magento): keep core features inside owned services to avoid security and performance issues from third-party extensions.

2.4 Feedback & Evaluation, Improvements, and Grading

Feedback & Evaluation

Lecturer's assessment of the project.

Item	Lecturer's Assessment
Overall evaluation	
Strengths	
Weaknesses	
Date	

Suggested Improvements

Areas where the project can be enhanced beyond the current scope:

- Add an API gateway with rate limiting and centralized authentication.

- Introduce asynchronous messaging (e.g., RabbitMQ or Kafka) for order and notification events.
- Add distributed tracing (e.g., OpenTelemetry/Jaeger) to follow requests across services.
- Implement a service mesh (e.g., Istio) for secure service-to-service communication.
- Add a Notification service (email/SMS) and a product review/rating service.
- Implement caching (e.g., Redis) for catalog and search results.
- Add a recommendation feature and an admin dashboard with sales analytics.
- Apply blue-green or canary deployments to reduce deployment risk.

Final Grading Criteria

Breakdown of marks based on documentation, implementation, testing, and presentation.

Category	What is assessed	Marks
Documentation	Planning, literature review, requirements, and architecture documents.	
Implementation	Working microservices, containerization, CI/CD, orchestration, IaC, and monitoring.	
Testing	Test plan, unit/integration/load tests, and reported results.	
Presentation	Live demo, clarity, and answers to questions.	
Total		

2.7 References

[1] Shopify. Shopify – Commerce platform. https://www.shopify.com

[2] Adobe. Adobe Commerce (Magento). https://business.adobe.com/products/magento/magento-commerce.html

[3] WooCommerce. WooCommerce – Open-source e-commerce plugin for WordPress. https://woocommerce.com

[4] Medusa. Medusa – Open-source commerce modules. https://medusajs.com

[5] Microsoft. eShopOnContainers – .NET microservices reference application. https://github.com/dotnet-architecture/eShopOnContainers

[6] Weaveworks. Sock Shop – Microservices demo application. https://github.com/microservices-demo/microservices-demo

[7] Newman, S. (2021). Building Microservices, 2nd Edition. O'Reilly Media.

[8] Richardson, C. (2018). Microservices Patterns. Manning Publications.

[9] Kubernetes Documentation. https://kubernetes.io/docs

[10] Prometheus Documentation. https://prometheus.io/docs

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Needs / Expectations	Interest in Project	Influence
Customers (shoppers)	Easy browsing and search, fast checkout, secure payment, order tracking.	High	High
Store administrators / sellers	Manage products, stock, and orders; view sales.	High	High
Development team	Clear requirements, maintainable code, automated pipelines.	High	High
DevOps / operations	Simple deployment, monitoring, alerting, easy rollback.	High	Medium
Payment gateway provider	Correct and secure API integration.	Medium	Medium
Cloud/hosting provider	Compliance with usage limits and terms.	Low	Low

3.2 User Stories & Use Cases

User Stories

ID	As a…	I want to… so that…	Priority
US1	Visitor	register and log in so that I can shop and keep my orders.	High
US2	Customer	browse products by category so that I can discover items.	High
US3	Customer	search for products by keyword so that I can find items quickly.	High
US4	Customer	view product details (price, images, stock) so that I can decide to buy.	High
US5	Customer	add, update, and remove items in my cart so that I can control my purchase.	High
US6	Customer	place an order from my cart so that I can buy the products.	High
US7	Customer	pay online securely so that I can complete my order.	High
US8	Customer	view my order history and status so that I know where my order is.	Medium
US9	Customer	update my profile and addresses so that checkout is faster.	Medium
US10	Administrator	add, edit, and delete products so that the catalog stays current.	High
US11	Administrator	view and manage orders so that I can fulfil them.	Medium
US12	DevOps engineer	see dashboards and alerts so that I can detect problems early.	High
US13	Developer	have code deployed automatically after merge so that releases are fast and safe.	High

Use Cases

Actors: Customer, Administrator, Payment Gateway (external), DevOps Engineer.

Use Case	Description
UC1 – Register / Login	Customer creates an account or logs in and receives an authentication token.
UC2 – Browse & Search Products	Customer views the catalog, filters by category, and searches by keyword.
UC3 – Manage Cart	Customer adds, updates, or removes items; the cart total is recalculated.
UC4 – Place Order & Pay	Customer checks out; the Order service creates an order and the Payment service charges via the gateway.
UC5 – Track Orders	Customer views past orders and their current status.
UC6 – Manage Products	Administrator creates, updates, or deletes products and stock.
UC7 – Monitor System	DevOps Engineer views metrics, logs, and alerts.

Detailed use case – UC4: Place Order & Pay

Field	Description
Actor	Customer (primary), Payment Gateway (secondary)
Preconditions	Customer is logged in and the cart contains at least one item.
Main flow	1. Customer selects “Checkout” and confirms the shipping address. 2. Order service validates the cart and product availability. 3. Order service creates the order with status “Pending”. 4. Payment service sends the payment request to the gateway. 5. Gateway confirms the payment. 6. Order status changes to “Paid” and the cart is cleared. 7. Customer sees an order confirmation.
Alternative flows	A. Item out of stock: the customer is informed and the cart is updated. B. Payment fails: the order stays “Pending/Failed” and the customer can retry.
Postconditions	Order is stored; payment is recorded; stock is updated.

Detailed use case – UC1: Register / Login

Field	Description
Actor	Visitor / Customer
Preconditions	The user has access to the website.
Main flow	1. User enters registration details. 2. User service validates data and checks that the email is unique. 3. Password is hashed and the account is saved. 4. User logs in with email and password. 5. User service returns a signed access token.
Alternative flows	A. Email already registered: an error message is shown. B. Wrong credentials: access is denied and the attempt is logged.
Postconditions	User has an authenticated session.

3.3 Functional Requirements

ID	Service	Requirement
FR1	User	The system shall allow users to register with email and password.
FR2	User	The system shall authenticate users and issue expiring access tokens.
FR3	User	The system shall allow users to view and edit their profile and addresses.
FR4	User	The system shall support at least two roles: Customer and Administrator.
FR5	Product	The system shall display a list of products with pagination and category filtering.
FR6	Product	The system shall display product details including name, description, price, images, and stock.
FR7	Product	Administrators shall be able to create, update, and delete products.
FR8	Search	The system shall allow searching products by keyword and filtering by price and category.
FR9	Cart	The system shall allow users to add, update the quantity of, and remove items in the cart.
FR10	Cart	The system shall calculate and display the cart total.
FR11	Order	The system shall create an order from the cart after validating stock.
FR12	Order	The system shall let users view their order history and order status.
FR13	Order	Administrators shall be able to view and update order status.
FR14	Payment	The system shall process payments through an external payment gateway (sandbox).
FR15	Payment	The system shall record the payment result and update the order status accordingly.
FR16	Platform	The system shall expose all services through a single API gateway/entry point.
FR17	Platform	Each service shall provide a health-check endpoint.
FR18	Platform	The CI/CD pipeline shall build, test, and deploy each service automatically on merge.
FR19	Platform	The system shall expose metrics and logs for every service for monitoring.

3.4 Non-functional Requirements

ID	Category	Requirement
NFR1	Performance	95% of API requests shall respond in under 500 ms under normal load.
NFR2	Performance	The system shall support at least 500 concurrent users during load testing.
NFR3	Scalability	Each service shall scale horizontally and independently (Kubernetes autoscaling).
NFR4	Availability	The system shall target 99.5% uptime; failed containers shall restart automatically.
NFR5	Reliability	A failure in one service shall not make unrelated functions unavailable (fault isolation, timeouts, retries).
NFR6	Security	Passwords shall be hashed; communication shall use HTTPS; secrets shall not be stored in source code.
NFR7	Security	Protected endpoints shall require valid tokens and enforce role-based access.
NFR8	Security	Card details shall never be stored by Marketly; the payment gateway handles them.

ID	Category	Requirement
NFR9	Usability	The web interface shall be responsive and usable on desktop and mobile browsers.
NFR10	Usability	A customer shall be able to complete a purchase in no more than five main steps.
NFR11	Maintainability	Services shall be independently deployable and have clear API documentation (OpenAPI).
NFR12	Portability	The system shall run identically on any machine through Docker containers.
NFR13	Observability	Metrics, logs, and alerts shall be available through centralized dashboards.
NFR14	Reproducibility	Environments shall be created from code (IaC) without manual steps.
NFR15	Data integrity	Each service shall own its database; database backups shall be available for recovery.